

Trabajo Sistemas Operativos 2:

parking.c práctica Windows

Sistemas Operativos II GIISI
USAL

Grupo: G08
Alumno 1: Brais Bértolo Senra
Alumno 2: Juan Riego Vila

Contenido

1. OBJETOS DE SINCRONIZACIÓN Y VALORES INICIALES	3
2. ZONA DE MEMORIA "COMPARTIDA" (VARIABLES GLOBALES).....	3
3. HILOS UTILIZADOS	3
4. PSEUDOCÓDIGO	4
4.1. LLEGADA_ALGORITMO(hc)	4
4.3. HILO_APARCAR.....	4
4.4. HILO_DESAPARCAR	4
4.5. APARCAR_COMMIT	4
4.7. PERMISO_AVANCE_COMMIT(hc)	5

1. OBJETOS DE SINCRONIZACIÓN Y VALORES INICIALES

- **hMutex[4]** → Valor inicial: libre. Exclusión mutua sobre `acera[]`, `carril[]` y `proxAparcar[]`. Solo un hilo por algoritmo puede modificar la memoria compartida a la vez.
- **hOrden[4]** → Valor inicial: 1. Garantiza que los `hilo_aparcar` de un mismo algoritmo aparquen en orden según el número de coche.
- **hAvance[4]** → Valor inicial: 0. Sincroniza el movimiento de los coches por el carril. Un hilo espera en `permiso_avance` si la posición destino está ocupada y después `permiso_avance_commit` lo despierta al liberar una posición.
- **hEventoFin** → Valor inicial: FALSE. Desbloquea el `WaitForSingleObject` del hilo principal e inicia el fin del programa de manera inmediata y correcta.

2. ZONA DE MEMORIA "COMPARTIDA" (VARIABLES GLOBALES)

Al usar hilos dentro de un mismo proceso no se necesita memoria compartida como se necesitaba en Linux. Las siguientes variables globales cumplen esa función:

- **acera[4][80]** Valor inicial: todo a 0 (libre). Ocupación de las plazas de cada algoritmo. Contiene en cada posición o 0 si está libre o la longitud del coche que la ocupa si está ocupada.
- **carril[4][80]** Valor inicial: todo a 0 (libre). Como el anterior pero indica la ocupación de posiciones en el carril.
- **proxAparcar[4]** Valor inicial: 0. Puntero para el algoritmo Siguiente Ajuste que indica donde debe de aparcar el siguiente coche.
- **proxAparcarNum[4]** Valor inicial: 1. Número del próximo coche al que le toca aparcar en cada algoritmo, en este caso empieza el 1 ya que se usa el orden de número ascendente.

3. HILOS UTILIZADOS

- **Hilo principal:** Ejecuta `parking()`. Arranca la simulación y gestiona el cierre.
- **hilo_aparcar:** Creado por `llegada_ajuste` (pudiendo ser ajuste cada uno de los cuatro ajustes) para cada coche que encuentra hueco en el aparcamiento.
- **hilo_desaparcar:** creado dinámicamente por `salida_ajuste` (pudiendo ser ajuste cada uno de los cuatro ajustes) para cada coche que sale del aparcamiento.
- **CtrlHandler:** hilo que señaliza `hEventoFin` al recibir `Ctrl+C`.

4. PSEUDOCÓDIGO

4.1. LLEGADA_ALGORITMO(hc)

```
llegada_algoritmo_X(hc)
{
    Lock(hMutex[alg])
    pos = buscar_hueco(alg)
    escribir_acera(pos, longitud)
    Unlock(hMutex[alg])
    Si (pos >= 0)
        CreateThread(hilo_aparcar, hc)
    Retornar pos
}
```

4.2. SALIDA_ALGORITMO(hc)

```
salida_algoritmo_X(hc)
{
    CreateThread(hilo_desaparcar, hc)
    Retornar 0
}
```

4.3. HILO_APARCAR

```
hilo_aparcar(hc)
{
    alg = getAlgoritmo(hc)

    // Espera turno estricto por numero de coche
    Por_siempre_jamas
    {
        W(hOrden[alg])
        Lock(hMutex[alg])
        mi_turno = (getNumero(hc) == proxAparcarNum[alg])
        Unlock(hMutex[alg])

        Si (mi_turno) break
        Sino
            Signal(hOrden[alg])
            Sleep(5ms)
    }
    PARKING2_aparcar(hc, aparcar_commit, permiso_avance, permiso_avance_commit)
}
```

4.4. HILO_DESAPARCAR

```
hilo_desaparcar(hc)
{
    PARKING2_desaparcar(hc, permiso_avance, permiso_avance_commit)
}
```

4.5. APARCAR_COMMIT

```
aparcar_commit(hc)
{
    alg = getAlgoritmo(hc)
    Lock(hMutex[alg])
    proxAparcarNum[alg]++
    Unlock(hMutex[alg])
    Signal(hOrden[alg])
}
```

4.6. PERMISO_AVANCE

```
permiso_avance(hc)
{
// Caso 1: avance por el carril (Y1=Y2=2)
Por_siempre_jamas
{
Lock(hMutex[alg])

Si (carril[alg][X2] == 0)
    carril[alg][X2] = 1
    Unlock(hMutex[alg])
    break

Unlock(hMutex[alg])
W(hAvance[alg])
}

// Caso 2: salida de acera al carril (Y1 < Y2 = 2)
Por_siempre_jamas
{
Lock(hMutex[alg])

Si (celdas X2 .. X2+longitud-1 libres en carril)
    ocupar_celdas()
    Unlock(hMutex[alg])
    break

Unlock(hMutex[alg])
W(hAvance[alg])

}
}
```

4.7. PERMISO_AVANCE_COMMIT(hc)

```
permiso_avance_commit(hc)
{
// Coche paso de acera (Y=1) al carril (Y=2): libera su plaza de acera
Si (Y_anterior==1 y Y_actual==2)
Lock(hMutex[alg])
vaciar_pos_acera(posEnAcera, longitud, alg)
Unlock(hMutex[alg])

// Coche paso del carril (Y=2) a la acera (Y=1): libera celdas de carril
Si (Y_anterior==2 y Y_actual==1)
Lock(hMutex[alg])
carril[alg][X_ant .. X_ant+long-1] = 0
Unlock(hMutex[alg])
Signal(hAvance[alg])

// Coche avanzo una celda en el carril (Y=2 -> Y=2): libera cola trasera
Si (Y_anterior==2 y Y_actual==2)
Lock(hMutex[alg])
carril[alg][X_ant + longitud - 1] = 0
Unlock(hMutex[alg])
Signal(hAvance[alg])
}
```